Roadmap HelpDesk IT – plan de atac pas cu pas

Proiect: HelpDesk IT – sistem web de gestionare a tichetelor de suport tehnic
Client (fictiv): TechNova Solutions SRL
Manager de proiect / dezvoltator: Văduva Alex
Perioada: 22.10.2026 – 15.01.2027
Stack: Python 3.12, Flask, SQLAlchemy, SQLite, Bootstrap 5, pytest, Ruff, GitHub Actions, Docker, Render

---

1. Ideea în două fraze

Proiectul e împărțit în **36 de pași mici**. Fiecare pas este un **issue** pe GitHub și lasă urme verificabile (branch, commit-uri, pull request, teste), deci profesorul poate urmări tot drumul de la cerință la cod. Asta este **trasabilitatea** pe care o punctează.

2. Dicționar rapid

Termen	Ce înseamnă
Issue	O sarcină pe GitHub, cu număr (ex. #17). Fiecare pas din acest roadmap e un issue.
Milestone	Un grup de issue-uri cu termen comun – aici, câte unul pentru fiecare raport.
Board (Kanban)	Tabla cu coloanele Todo / In progress / Done, unde muți issue-urile.
Branch	O copie separată a codului, unde lucrezi la un singur issue fără să strici `main`.
Commit	O „salvare” a modificărilor, cu un mesaj care spune ce ai făcut și la ce issue.
Pull Request (PR)	Cererea de a aduce branch-ul în `main`; aici se văd modificările și rezultatul testelor.
CI	GitHub Actions rulează automat linterul și testele la fiecare PR (bifă verde/roșie).
Tag	O etichetă pusă pe versiunea predată (ex. `v0.1-propunere`).

3. Regula de aur – bucla repetată la FIECARE pas

1. Iei un issue din coloana **Todo** și îl muți în **In progress**.
2. Creezi un branch: `git switch -c feature/17-creare-tichet` (numărul issue-ului + nume scurt).
3. Lucrezi și faci **commit-uri mici**, cu numărul issue-ului: `git commit -m "feat: formular creare tichet (#17)"`.
4. Trimiți branch-ul: `git push -u origin feature/17-creare-tichet`.
5. Deschizi un **Pull Request** și scrii în descriere `Closes #17`.
6. Aștepți bifa verde de la **CI**. Dacă e roșie, repari și faci push din nou.
7. **Merge** în `main` → issue-ul se închide singur → cardul trece în **Done**.
8. Revii pe main: `git switch main && git pull`.

Convenții

- Mesaje de commit: `feat:` funcționalitate nouă · `fix:` reparație · `docs:` documentație · `test:` teste · `ci:` integrare continuă · `chore:` configurări.
- Label-uri: `cerință`, `documentație`, `infrastructură`, `bug` + prioritate MoSCoW: `must` (obligatoriu), `should` (de dorit), `could` (bonus).
- Câte un **tag** la fiecare predare: `v0.1-propunere`, `v0.2-mvp`, `v1.0`.

4. Calendarul pe scurt

Milestone	Perioada	Issues	Ce predai
0 – Inițiere	21.10 – 23.10	#1 – #2	Carta, minuta, raport preliminar
1 – Raport 1: Propunere	24.10 – 01.11	#3 – #8	Specificații, tehnologii, diagrama cazurilor de utilizare

Milestone	Perioada	Issues	Ce predai
2 – Raport 2: Proiectare și MVP	02.11 – 29.11	#9 – #21	Diagrame de proiectare + aplicație minimală (login, tichete)
3 – Final	30.11 – 15.01	#22 – #36	Aplicația completă, Docker, deploy, documentație, prezentare

Tampon pentru întârzieri: #27 (filtrare) și #31 (email) sunt opționale – se taie primele dacă rămâi în urmă.

Milestone 0 – Inițiere (termen 23.10.2026)

Carta proiectului, minuta ședinței, raportul preliminar

#1 – Carta proiectului şi minuta şedinţei de iniţiere

Când: 22–23 oct · **Label-uri:** `documentație` `must`

Ce faci:

- Ședința de inițiere cu clientul (minuta)
- Completarea cartei proiectului pe șablonul cursului
- Urcarea fișierelor în `docs/`

Gata când: `docs/Carta proiectului - HelpDesk IT.pdf` și `docs/Minuta intalnirii - HelpDesk IT.pdf` există în repository.
De ce: Carta „autorizează” proiectul: scop, obiective, echipă, termene, buget. Toate celelalte documente pornesc de la ea.

#2 – Raport preliminar: Tehnologii şi instrumente pentru dezvoltarea programelor

Când: 21–23 oct · **Label-uri:** `documentație` `must`

Ce faci:

- Redactare raport (introducere, istoric, categorii, concluzii, referințe)
- Export PDF și încărcare pe Classroom
- Urcare în `docs/`

Gata când: Raportul e predat și se află în `docs/`.
De ce: Primul livrabil notat (20%). Justifică alegerea uneltelor folosite în proiect.

Milestone 1 – Raport 1: Propunere (termen 01.11.2026)

Specificații, tehnologii, diagrama cazurilor de utilizare

#3 – Creează repository-ul GitHub

Când: 24 oct · **Label-uri:** `infrastructură` `must`

Ce faci:

- ☐ Pe github.com: New repository → `helpdesk-it`, bifează „Add README”, `.gitignore` = Python, licență MIT
- ☐ `git clone` pe calculator
- ☐ Copiază folderul `docs/` și fișierul `ROADMAP.md`
- ☐ `git add . && git commit -m "docs: carta, minuta, raport preliminar (#1 #2)"` și `git push`
- ☐ Invită clientul/profesorul ca colaboratori (Settings → Collaborators) – acțiune din minută

Gata când: Repository-ul e vizibil online și conține documentele de inițiere.
De ce: Repository-ul este „sursa adevărului”: tot ce faci de acum lasă urme aici.

#4 – Configurează board-ul Kanban, label-urile şi milestone-urile

Când: 24–25 oct · **Label-uri:** `infrastructură` `must`

Ce faci:

- ☐ Instalează GitHub CLI (`gh`) și rulează `gh auth login`
- ☐ Rulează `bash scripts/setup_github.sh <utilizator>/helpdesk-it` (creează label-uri, milestones și toate issue-urile)
- ☐ Pe GitHub: Projects → New project → Board, cu coloanele Todo / In progress / Done
- ☐ Adaugă toate issue-urile în board (Add items → selectează repository-ul)
- ☐ În setările proiectului activează workflow-urile „Item closed → Done" și „Pull request merged → Done"

Gata când: Board-ul arată toate issue-urile, fiecare cu milestone și label-uri.
De ce: Board-ul este planul vizibil al proiectului; profesorul vede progresul fără să întrebe.

#5 – Specificațiile aplicației (cerințe funcționale și nefuncționale)

Când: 26–28 oct · **Label-uri:** `cerință` `documentație` `must`

Ce faci:

- ☐ Creează `docs/specificatii.md`
- ☐ Scrie actorii: Angajat, Tehnician IT, Manager IT
- ☐ Scrie cerințele funcționale F1–F12 (ex.: F1 Autentificare, F2 Creare tichet, F3 Lista tichetelor mele …)
- ☐ Scrie cerințele nefuncționale (pagini sub 2 s, parole criptate, date de test fictive – GDPR, interfață în română)
- ☐ Commit `docs: specificatii (#5)` → PR → merge

Gata când: Fiecare cerință are un cod (F1, F2 …) folosit mai departe în issues și teste.
De ce: Codurile cerințelor sunt baza trasabilității: cerință → issue → cod → test.

#6 – Diagrama cazurilor de utilizare (PlantUML)

Când: 28–29 oct · **Label-uri:** `documentație` `must`

Ce faci:

- ☐ Creează `docs/diagrame/use-case.puml`
- ☐ Desenează cei 3 actori și cazurile de utilizare (Deschide tichet, Preia tichet, Atribuie tichet, Schimbă status, Comentează, Vezi statistici, Administrează utilizatori)
- ☐ Generează imaginea (extensia PlantUML din VS Code sau plantuml.com) → `use-case.png`
- ☐ Commit `docs: diagrama cazurilor de utilizare (#6)`

Gata când: Diagrama acoperă toate cerințele F1–F12.
De ce: Cerută explicit în Raportul 1. Fiind text, diagrama e versionată ca și codul.

#7 – Descrierea tehnologiilor folosite

Când: 29–30 oct · **Label-uri:** `documentație` `must`

Ce faci:

- ☐ Listează tehnologiile: Python, Flask, SQLAlchemy, SQLite, Bootstrap, pytest, Ruff, GitHub Actions, Docker, Render
- ☐ Pentru fiecare: o frază „ce este" + o frază „de ce l-am ales"
- ☐ Poți refolosi tabelul 3 din raportul preliminar

Gata când: Secțiunea „Tehnologii" e scrisă.
De ce: Cerută explicit în Raportul 1.

#8 – Redactează și predă Raportul 1

Când: 30 oct – 1 nov · **Label-uri:** `documentație` `must`

Ce faci:

- ☐ Reunește specificațiile, tehnologiile și diagrama într-un document
- ☐ Adaugă o captură de ecran cu board-ul GitHub (dovada de trasabilitate)
- ☐ Export PDF în `docs/` → commit → predare pe Classroom
- ☐ Creează tag-ul: `git tag v0.1-propunere && git push --tags`

Gata când: Raportul 1 e predat, tag-ul `v0.1-propunere` există.
De ce: Tag-ul marchează exact starea proiectului la predare.

Milestone 2 – Raport 2: Proiectare și MVP (termen 29.11.2026)

Diagrame de proiectare + implementare minimală funcțională

#9 – Modelul bazei de date (ERD)

Când: 2–4 nov · **Label-uri:** `documentație` `must`

Ce faci:

- ☐ Tabelele: `User` (nume, email, parolă hash, rol), `Category`, `Ticket` (titlu, descriere, prioritate, status, autor, tehnician, categorie, date), `Comment`, `TicketHistory`
- ☐ Desenează relațiile (un User are mai multe Ticket etc.) în `docs/diagrame/erd.puml`
- ☐ Commit `docs: ERD (#9)`

Gata când: Diagrama ERD conține cele 5 tabele și relațiile lor.
De ce: Baza de date se proiectează înainte de cod, ca să nu o refaci ulterior.

#10 – Diagrama de arhitectură și structura folderelor

Când: 4–6 nov · **Label-uri:** `documentație` `must`

Ce faci:

- ☐ Desenează: Browser → Flask (rute + șabloane) → SQLAlchemy → SQLite
- ☐ Scrie structura folderelor din ROADMAP în `docs/arhitectura.md`
- ☐ Commit `docs: arhitectura (#10)`

Gata când: Documentul explică pe ce strat stă fiecare fișier.
De ce: Necesară pentru partea de „proiectare” din Raportul 2.

#11 – Machete (wireframes) pentru ecranele principale

Când: 6–8 nov · **Label-uri:** `documentație` `should`

Ce faci:

- ☐ În draw.io sau pe hârtie: Login, Lista tichetelor, Tichet nou, Detaliu tichet, Dashboard
- ☐ Salvează imaginile în `docs/machete/`
- ☐ Commit `docs: machete interfata (#11)`

Gata când: 5 machete în `docs/machete/`.
De ce: Arăți clientului cum va arăta aplicația înainte s-o construiești.

#12 – Scheletul aplicației Flask

Când: 9–10 nov · **Label-uri:** `infrastructură` `must`

Ce faci:

- ☐ `python -m venv .venv` și activează mediul virtual
- ☐ `pip install flask flask-sqlalchemy flask-login` → `pip freeze > requirements.txt`
- ☐ Creează `app/__init__.py` cu funcția `create_app()` și o rută `/` care afișează „HelpDesk IT”
- ☐ Rulează `flask --app app run` și deschide http://127.0.0.1:5000
- ☐ Commit `feat: schelet aplicatie Flask (#12)`

Gata când: Pagina de start se deschide în browser.
De ce: Primul cod care rulează; toate funcționalitățile se vor adăuga peste el.

#13 – Linter (Ruff) și teste (pytest) – primul test

Când: 10–11 nov · **Label-uri:** `infrastructură` `must`

Ce faci:

- ☐ `pip install ruff pytest` și adaugă-le în `requirements.txt`
- ☐ Creează `tests/test_home.py`: pagina `/` răspunde cu codul 200
- ☐ Rulează `ruff check .` și `pytest` – ambele trebuie să treacă
- ☐ Commit `test: primul test (#13)`

Gata când: `pytest` afișează „1 passed”.
De ce: Testele dovedesc că aplicația funcționează; linterul păstrează codul curat.

#14 – Integrare continuă cu GitHub Actions

Când: 11–12 nov · **Label-uri:** `infrastructură` `must`

Ce faci:

- ☐ Creează `.github/workflows/ci.yml`: la fiecare push/PR instalează Python, rulează `ruff check .` și `pytest`
- ☐ Push → verifică tab-ul „Actions” (bifă verde)
- ☐ Adaugă badge-ul CI în README
- ☐ Commit `ci: workflow lint + teste (#14)`

Gata când: Fiecare PR afișează automat bifa verde/roșie.
De ce: De acum, nicio modificare nu intră în `main` fără teste trecute – dovadă clară de calitate.

#15 – Modelele bazei de date și datele de test

Când: 12–15 nov · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ Scrie în `app/models.py` clasele `User`, `Category`, `Ticket` după ERD (#9)
- ☐ Creează `seed.py`: 3 utilizatori (câte unul pe rol), 4 categorii (hardware, software, rețea, conturi)
- ☐ Rulează `python seed.py` și verifică datele
- ☐ Commit `feat: modele + seed (#15)`

Gata când: Baza `helpdesk.db` conține utilizatorii și categoriile de test.
De ce: Datele fictive permit testarea fără date personale reale (GDPR – constrângere din cartă).

#16 – Autentificare (login / logout) – F1

Când: 16–17 nov · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ Pagina `/login` cu email și parolă
- ☐ Parolele se verifică cu `check_password_hash` (nu se salvează niciodată în clar)

- ☐ Flask-Login ține minte utilizatorul; buton Logout
- ☐ Paginile interne sunt protejate cu `@login_required`
- ☐ Commit `feat: autentificare (#16)`

Gata când: Te poți loga cu un utilizator din seed; fără login ești redirecționat la `/login`.
De ce: Fiecare tichet trebuie să aibă un autor, deci trebuie să știm cine e logat.

#17 – Creare tichet – F2

Când: 18–19 nov · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ `git switch -c feature/17-creare-tichet`
- ☐ Pagina `/tickets/new`: titlu, descriere, categorie, prioritate (Scăzută / Medie / Ridicată / Critică)
- ☐ La salvare: status „Nou”, autor = utilizatorul logat
- ☐ Commit `feat: formular creare tichet (#17)`
- ☐ Test + commit `test: creare tichet (#17)` → PR „Closes #17”

Gata când: Tichetul creat apare în baza de date cu status „Nou”.
De ce: Funcționalitatea principală a aplicației.

#18 – Lista „Tichetele mele” – F3

Când: 20–21 nov · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ Pagina `/tickets` cu un tabel: număr, titlu, prioritate, status, dată
- ☐ Angajatul își vede doar tichetele lui
- ☐ Culori Bootstrap pentru prioritate (badge-uri)
- ☐ Commit `feat: lista tichete (#18)`

Gata când: Fiecare utilizator își vede doar propriile tichete.
De ce: Angajatul urmărește statusul cererilor sale – nevoia principală din minută.

#19 – Pagina de detaliu a tichetului – F4

Când: 21–22 nov · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ Ruta `/tickets/<id>` afișează toate câmpurile
- ☐ Un angajat nu poate deschide tichetul altcuiva (cod 403)
- ☐ Commit `feat: detaliu tichet (#19)`

Gata când: Click pe un tichet din listă deschide pagina lui.
De ce: Locul unde vor apărea ulterior statusul, istoricul și comentariile.

#20 – Teste pentru funcţionalităţile MVP

Când: 23–25 nov · **Label-uri:** `infrastructură` `must`

Ce faci:

- ☐ Fixture pytest cu o bază de date temporară
- ☐ Teste: login corect/greșit, creare tichet, „tichetele mele” nu arată tichetele altora, 403 pe tichet străin
- ☐ Commit `test: teste MVP (#20)`

Gata când: CI verde, minimum 6 teste.

De ce: Dovada că MVP-ul funcționează – se prezintă în Raportul 2.

#21 – Redactează și predă Raportul 2

Când: 26–29 nov · **Label-uri:** `documentație` `must`

Ce faci:

- ☐ Include: ERD, arhitectura, machetele, capturi de ecran din aplicație
- ☐ Secțiune „Trasabilitate”: captură board, listă de PR-uri, istoricul CI
- ☐ Export PDF în `docs/` → predare
- ☐ `git tag v0.2-mvp && git push --tags`

Gata când: Raportul 2 e predat, tag-ul `v0.2-mvp` există.
De ce: Al doilea livrabil intermediar (20%).

Milestone 3 – Final (termen 15.01.2027)

Aplicația completă, documentație, prezentare

#22 – Roluri și permisiuni – F5

Când: 30 nov – 2 dec · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ Decorator `role_required("tehnician", "manager")`
- ☐ Meniul afișează doar paginile permise rolului
- ☐ Teste pentru accesul interzis
- ☐ Commit `feat: roluri si permisiuni (#22)`

Gata când: Un angajat primește 403 pe paginile de tehnician/manager.
De ce: Cei 3 actori din diagrama cazurilor de utilizare au drepturi diferite.

#23 – Panoul tehnicianului: preluare tichete – F6

Când: 3–6 dec · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ Pagina `/tech`: tichetele neatribuite + tichetele mele
- ☐ Butonul „Preia” setează tehnicianul și statusul „În lucru”
- ☐ Commit `feat: panou tehnician (#23)`

Gata când: Un tehnician poate prelua un tichet nou.
De ce: Fluxul principal: cerere → preluare → rezolvare.

#24 – Managerul atribuie tichete – F7

Când: 7–9 dec · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ Pe pagina tichetului, managerul alege un tehnician dintr-o listă
- ☐ Test pentru atribuire
- ☐ Commit `feat: atribuire tichet (#24)`

Gata când: Managerul poate atribui orice tichet oricărui tehnician.
De ce: Cerință din minută: managerul distribuie munca.

#25 – Schimbare status și istoric – F8

Când: 10–13 dec · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ Statusuri: Nou → În lucru → Rezolvat → Închis
- ☐ Fiecare schimbare se salvează în `TicketHistory` (cine, când, din ce în ce)
- ☐ Istoricul se afișează pe pagina tichetului
- ☐ Commit `feat: status + istoric (#25)`

Gata când: Pagina tichetului arată cronologia completă.
De ce: Trasabilitate și în aplicație, nu doar în proces – obiectiv din cartă.

#26 – Comentarii pe tichet – F9

Când: 14–17 dec · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ Modelul `Comment` + formular pe pagina tichetului
- ☐ Afișare: autor, dată, text
- ☐ Commit `feat: comentarii (#26)`

Gata când: Angajatul și tehnicianul pot discuta pe tichet.
De ce: Înlocuiește emailurile și telefoanele pierdute (problema clientului).

#27 – Filtrare și căutare tichete – F10

Când: 18–20 dec · **Label-uri:** `cerință` `should`

Ce faci:

- ☐ Filtre după status, prioritate, categorie
- ☐ Căutare în titlu
- ☐ Commit `feat: filtrare si cautare (#27)`

Gata când: Lista se filtrează corect.
De ce: Opțional (should): dacă apar întârzieri, se amână primul.

#28 – Dashboard cu statistici – F11

Când: 21–28 dec · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ Pagina `/dashboard` doar pentru manager
- ☐ Grafice Chart.js: tichete pe categorie, pe status, pe tehnician
- ☐ Commit `feat: dashboard (#28)`

Gata când: Managerul vede 3 grafice cu date reale din baza de date.
De ce: Obiectiv din cartă: rapoarte pentru managerul IT.

#29 – Alertă SLA – tichete fără reacție peste 4 ore

Când: 29 dec – 3 ian · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ Funcție care găsește tichetele „Nou” mai vechi de 4 ore
- ☐ Evidențiere cu roșu în listă și pe dashboard

- ☐ Test cu un tichet „îmbătrânit” artificial
- ☐ Commit `feat: alerta SLA (#29)`

Gata când: Tichetele întârziate apar marcate.
De ce: Obiectiv din cartă: primă reacție sub 4 ore lucrătoare.

#30 – Administrare utilizatori și categorii – F12

Când: 4–6 ian · **Label-uri:** `cerință` `must`

Ce faci:

- ☐ Pagini pentru manager: listă, adăugare, dezactivare utilizatori; listă/adăugare categorii
- ☐ Commit `feat: administrare (#30)`

Gata când: Managerul poate adăuga un utilizator nou care se poate loga.
De ce: Clientul trebuie să poată folosi aplicația fără programator.

#31 – Notificări email (opțional)

Când: 6–7 ian · **Label-uri:** `cerință` `could`

Ce faci:

- ☐ Flask-Mail: email la atribuire și la rezolvare
- ☐ În dezvoltare, emailurile se afișează în consolă
- ☐ Commit `feat: notificari email (#31)`

Gata când: Atribuirea unui tichet generează un email.
De ce: Bonus (could) – primul tăiat dacă timpul nu ajunge.

#32 – Docker: Dockerfile și docker-compose

Când: 7–8 ian · **Label-uri:** `infrastructură` `must`

Ce faci:

- ☐ `Dockerfile` bazat pe `python:3.12-slim`, pornire cu gunicorn
- ☐ `docker-compose.yml` cu volum pentru baza de date
- ☐ `docker compose up` → aplicația merge pe http://localhost:8000
- ☐ Commit `chore: docker (#32)`

Gata când: Aplicația pornește identic pe orice calculator cu Docker.
De ce: Planul B pentru demo, conform riscului din cartă (Render poate „adormi”).

#33 – Deploy pe Render

Când: 9–10 ian · **Label-uri:** `infrastructură` `must`

Ce faci:

- ☐ Cont Render → New Web Service → conectează repository-ul
- ☐ Setează comanda de pornire și variabila `SECRET_KEY`
- ☐ Pune link-ul live în README
- ☐ Commit `docs: link aplicatie live (#33)`

Gata când: Aplicația e accesibilă online.
De ce: Livrabil din cartă: „cod GitHub + link live”.

#34 – Testare finală și corectarea erorilor

Când: 11–12 ian · **Label-uri:** `bug` `must`

Ce faci:

- ☐ Parcurge toate cazurile de utilizare cu fiecare rol
- ☐ Fiecare problemă găsită → issue nou cu label `bug`
- ☐ Rezolvă bug-urile pe branch-uri `fix/…`

Gata când: Niciun bug deschis cu label `must`; CI verde.
De ce: Bug-urile documentate ca issues sunt și ele trasabilitate.

#35 – Documentație și manual de utilizare

Când: 12–13 ian · **Label-uri:** `documentație` `must`

Ce faci:

- ☐ README: descriere, instalare, rulare, testare, Docker, link live
- ☐ `docs/manual-utilizare.md` cu capturi de ecran pentru fiecare rol
- ☐ Commit `docs: manual (#35)`

Gata când: Un om nou poate instala și folosi aplicația doar citind documentația.
De ce: Face parte din „predarea către client”.

#36 – Raport final și prezentare

Când: 13–15 ian · **Label-uri:** `documentație` `must`

Ce faci:

- ☐ Raport final: rezumat, cerințe vs. realizat, diagrame, trasabilitate (statistici din GitHub: issues, PR-uri, commit-uri)
- ☐ Prezentare de 10 minute + demo live
- ☐ `git tag v1.0 && git push --tags`

Gata când: Proiect predat, tag `v1.0`.
De ce: 40% din nota finală.

Structura proiectului (la final)

```
helpdesk-it/
├── app/
│   ├── __init__.py        # create_app()
│   ├── models.py          # User, Category, Ticket, Comment, TicketHistory
│   ├── auth/              # login / logout            (#16)
│   ├── tickets/           # tichete, comentarii       (#17–#19, #23–#27)
│   ├── admin/             # utilizatori, categorii, dashboard (#28–#30)
│   ├── templates/         # pagini HTML (Jinja2 + Bootstrap)
│   └── static/
├── tests/                 # teste pytest              (#13, #20 …)
├── docs/                  # rapoarte, specificații, diagrame
├── scripts/setup_github.sh
├── .github/workflows/ci.yml   (#14)
├── Dockerfile, docker-compose.yml (#32)
├── requirements.txt, pyproject.toml
├── seed.py                (#15)
├── README.md
└── ROADMAP.md
```

Cum arată trasabilitatea la final (de pus în rapoarte)

- **Cerință → issue:** F2 „Creare tichet” = issue #17.
- **Issue → cod:** commit-urile „(#17)” și PR-ul „Closes #17”.
- **Cod → test:** `tests/test_tickets.py` rulat de CI în PR-ul #17.
- **Progres:** board-ul Kanban + milestones (procent de finalizare) + tag-urile fiecărei predări.